Full Stack Development using MERN

Digital Record System for Community Health Workers

Student Name	Ayush Rajput
Roll Number	T036
Class	TYBScIT
Subject	Full Stack Development using MERN
Academic Year	2026-2027
College Name	Sheth L.U.J. College of Arts and Sir M.V. College of Science and Commerce
Department Name	Information Technology
Submitted to	Prof. Rohini Bhosale

1. Project Title

Digital Record System for Community Health Workers

2. Objective

The main objective of the Digital Record System for Community Health Workers is to develop a web-based application for maintaining and managing community health records digitally. The system helps community health workers store personal information, maintain health-related details, schedule follow-up appointments, and track follow-up statuses.

The project aims to reduce manual record-keeping, improve the organization of health information, and provide easy access to registered records through a simple web interface. It also demonstrates database connectivity, user authentication, form validation, session management, and CRUD operations.

3. Project Description

The Digital Record System for Community Health Workers is a web-based application developed to maintain personal and health-related information in a centralized database.

The application provides a login facility through which registered users can access the system. After successful login, users can view a dashboard containing the total number of registered people, pending follow-ups, upcoming follow-ups, and recently registered records.

The People Management module allows users to register new individuals, search existing records, view personal details, and edit information. The stored information includes full name, date of birth, gender, phone number, address, blood group, allergies, health issues, and emergency contact details.

The Follow-up Management module allows users to schedule follow-ups for registered individuals. Users can view follow-up dates, reasons, and statuses. The available statuses are Pending, Completed, and Missed.

The application uses Python and Flask for backend processing, MySQL for database management, and HTML, CSS, and JavaScript for the web interface. The MySQL Connector enables communication between the Python application and the database.

The system also includes password hashing, session-based authentication, server-side validation, parameterized SQL queries, and basic CSRF token checks.

4. Functional Requirements

Feature	Requirements
User Login	Allows registered and active users to log in using their email address and password.
User Authentication	Verifies user credentials and maintains a session after successful login.
Dashboard	Displays the total number of registered people, pending follow-ups, upcoming follow-ups, and recently registered records.
Register Person	Allows users to add personal and health-related information to the database.
View Records	Displays registered individuals and their basic information.
Search Records	Allows users to search for individuals using their name, phone number, or record ID.
View Person Details	Displays complete available details of a selected individual, including associated follow-ups.
Edit Records	Allows users to update personal and health-related information.
Add Follow-up	Allows users to schedule a follow-up date and enter its reason.

View Follow-ups	Displays scheduled follow-ups along with the person's name, date, reason, and status.
Update Follow-up Status	Allows users to change the status to Pending, Completed, or Missed.
User Logout	Ends the current session and redirects the user to the login page.
Error Handling	Displays suitable messages when login, database operations, or form submissions fail.

5. Tools and Technologies Used

- **Python:** Used for backend programming and application logic.
- **Flask:** Used as the web application framework for routing and request handling.
- **MySQL:** Used to store user accounts, personal records, and follow-up information.
- **MySQL Connector/Python:** Connects the Python application to the MySQL database.
- **HTML:** Used to structure the web pages and forms.
- **CSS:** Used to style the website interface.
- **JavaScript:** Used for client-side interactions where implemented.
- **Werkzeug:** Used for secure password hashing and password verification.
- **Python-dotenv:** Used to load configuration settings from environment variables.
- **Visual Studio Code:** Used for writing and editing project code.
- **Git and GitHub:** Used for source-code version control and repository hosting.

6. Technical Requirements

Component	Requirement
Frontend	HTML and CSS are used to create the website pages, forms, tables, and layout.

Backend	Python and Flask handle application routes, form submissions, and database operations.
Database	MySQL stores worker accounts, personal records, and follow-up details.
Database Connectivity	MySQL Connector/Python establishes the connection between Flask and MySQL.
Authentication	Email and password verification is used to authenticate registered users.
Password Security	Werkzeug password hashing functions are used to store and verify password hashes.
Session Management	Flask sessions maintain the logged-in user's identity and role.
Database Operations	SQL INSERT, SELECT, and UPDATE statements are used to manage records.
Search Functionality	SQL queries search records using names, phone numbers, and record IDs.
Form Validation	Server-side checks validate required names, blood groups, dates, phone numbers, and follow-up statuses.
Security Measures	Parameterized SQL queries and basic session-bound CSRF token checks are implemented.

Configuration	Environment variables are used to configure database connection details and the Flask secret key.
Error Handling	Database errors are logged and appropriate messages are displayed to users.

7. Validation Requirements

- **Full Name Validation:** The person's full name must not be empty.
- **Blood Group Validation:** Only the permitted blood group values are accepted when a blood group is provided.
- **Date of Birth Validation:** The date of birth must follow a valid date format when entered.
- **Phone Number Validation:** The phone number must not exceed the defined length limit.
- **Follow-up Date Validation:** A follow-up date is required and must be a valid date.
- **Follow-up Status Validation:** Only Pending, Completed, and Missed statuses are accepted.
- **Login Validation:** The email and password are checked against an active user's stored credentials.
- **Session Validation:** Protected pages require an authenticated session.
- **CSRF Validation:** POST requests are checked for a valid session-bound CSRF token.
- **Database Validation:** Database errors are handled, and unsuccessful changes are rolled back where implemented.
- **Search Validation:** Search input is processed to find matching names, phone numbers, or record IDs.
- **Configuration Security:** Database credentials and the Flask secret key should be maintained in environment configuration rather than exposed in public source code.

8. Testing and Demonstration

The application should be tested to verify that its major features work correctly.

Test Case 1: User Login

Enter valid login credentials and verify that the user is redirected to the dashboard. Enter incorrect credentials and verify that an error message is displayed.

Test Case 2: Dashboard

Open the dashboard and verify that registered people, pending follow-ups, upcoming follow-ups, and recent records are displayed.

Test Case 3: Register a Person

Enter the required personal and health-related details and save the form. Verify that the new record is stored in the database.

Test Case 4: Search Records

Search for a registered person using their name, phone number, or record ID. Verify that matching records are displayed.

Test Case 5: View and Edit Records

Open a person's details, modify the information, and save the changes. Verify that the updated information is displayed correctly.

Test Case 6: Add Follow-up

Select a registered person, enter a valid follow-up date and reason, and verify that the follow-up is saved.

Test Case 7: Update Follow-up Status

Change a follow-up status to Pending, Completed, or Missed and verify that the updated status appears in the follow-up list.

Test Case 8: Form Validation

Submit forms with missing or invalid values and verify that the application displays appropriate validation messages.

Test Case 9: Logout

Log out of the application and verify that the session ends and the login page is displayed.

Test Case 10: Database Connectivity

Verify that the application can connect to MySQL and that records are saved and retrieved correctly.

For the final demonstration, use fictional sample data rather than real patient information. The project is intended for academic demonstration and should not be used to store real patient records without appropriate security and privacy review.

9. GitHub Repository and Submission

The project source code is maintained in a GitHub repository. The repository contains the Flask application, database schema, configuration example, dependency list, and administrator account creation script.

GitHub Repository Link:

https://github.com/ayushrajputmvlu-stack/Mern-Project/upload

Important Project Files:

- `app.py` – Contains Flask routes, login and logout logic, dashboard processing, record management, follow-up management, validation, and database operations.
- `database.sql` – Creates the `community_health_db` database and the `workers`, `people`, and `followups` tables.
- `create_admin.py` – Creates the initial administrator account with a hashed password.
- `requirements.txt` – Lists the Python packages required to run the application.
- `.env.example` – Provides an example of the environment variables required for database configuration.
- `README.md` – Contains project information, setup instructions, and troubleshooting guidance.
- `style.css` – Contains website styling, where included in the repository.

Conclusion

The Digital Record System for Community Health Workers provides a structured way to manage personal records and follow-up appointments through a web-based application. It demonstrates the practical use of Python, Flask, MySQL, database connectivity, user authentication, form validation, and record management.

The project helps organize information digitally and provides a foundation for further improvements in community health record management.